青 岛 科 技 大 学

机器学习大作业报告

题 目 智慧金融及其数据挖掘

指导教师 刘金环

学生姓名 赵宸

学生学号 2302020530

化学与分子工程 **院（部）** 化学 **专业** 化学 235 **班**

2025 年 2 月 16 日

1.设计背景

随着社会经济的高速发展，技术进步以及市场需求的进一步扩大，对智慧金融的要求也比以往更加严苛了。随着人工智能，大数据，云计算等技术的发展，金融行业的工作人员对于数据的依赖程度更大了，同时，金融行业在业务流程、业务开拓和客户服务等方面的智能化也更加深刻。而且智慧金融的发展也得到了政府的大力支持，所以在综上所述的各种条件下，为了通过庞大的银行信息，帮助银行工作人员筛选潜在的投资人群，我们设计了一种算法，帮助银行对消费群体进行合理的聚类。

智慧金融的意义无疑是非常巨大的，在海量信息的时代下，计算机的高算力使得那些人工无法解决的庞大数据的归纳变得可能，而且对于高维数据，人类是无法进行可视化的，但是计算机可以。所以通过智慧金融的搭建，可以极大程度的解放人力，更好的归纳整合数据，为金融行业的进一步发展提供有效的帮助。

2.K-Means 算法概述

2.1 K-Means 算法提出的背景

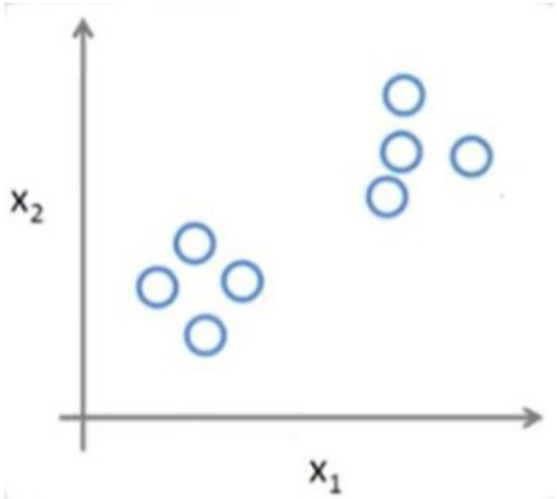
K-Means 算法是在 20 世纪 50 年代和 60 年代由不同领域的研究者独立提出的。该算法提出的背景与当时数据分析需求的增长有关，研究者们需要一种方法来自动的将数据集划分为不同的组或簇，以便更好地理解和分析数据。

该算法以其简单性和低计算复杂度而被广泛接受，在大数据时代该算法将发挥更大的优势，能够快速将相似的数据划分到同一个簇中

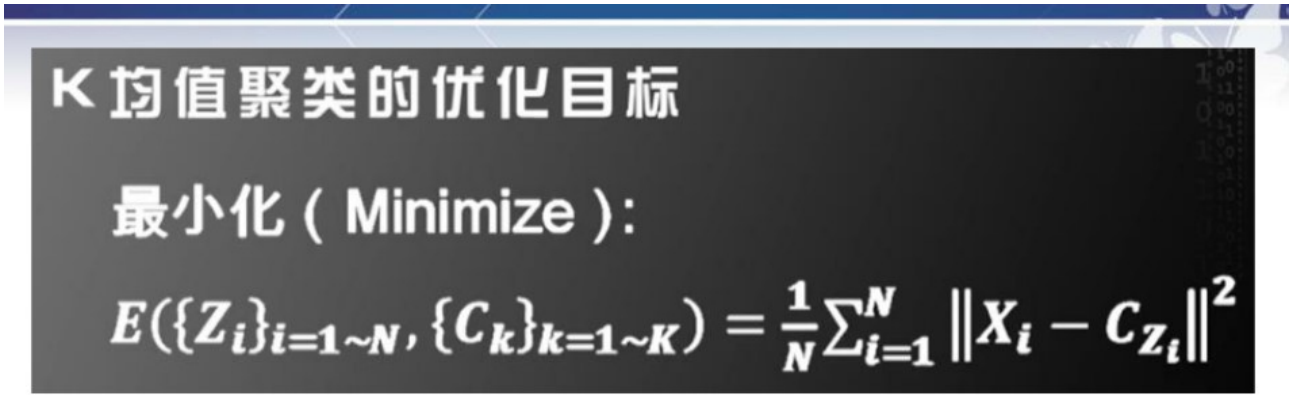
2.2 K-Means 算法的基本原理

K-Means 算法是一种迭代优化算法，其基本原理是将数据集划分为 K 个簇，使得簇内的数据点相似度最大化，簇间的数据点相似度最小化，最大程度上分成不同的组别。

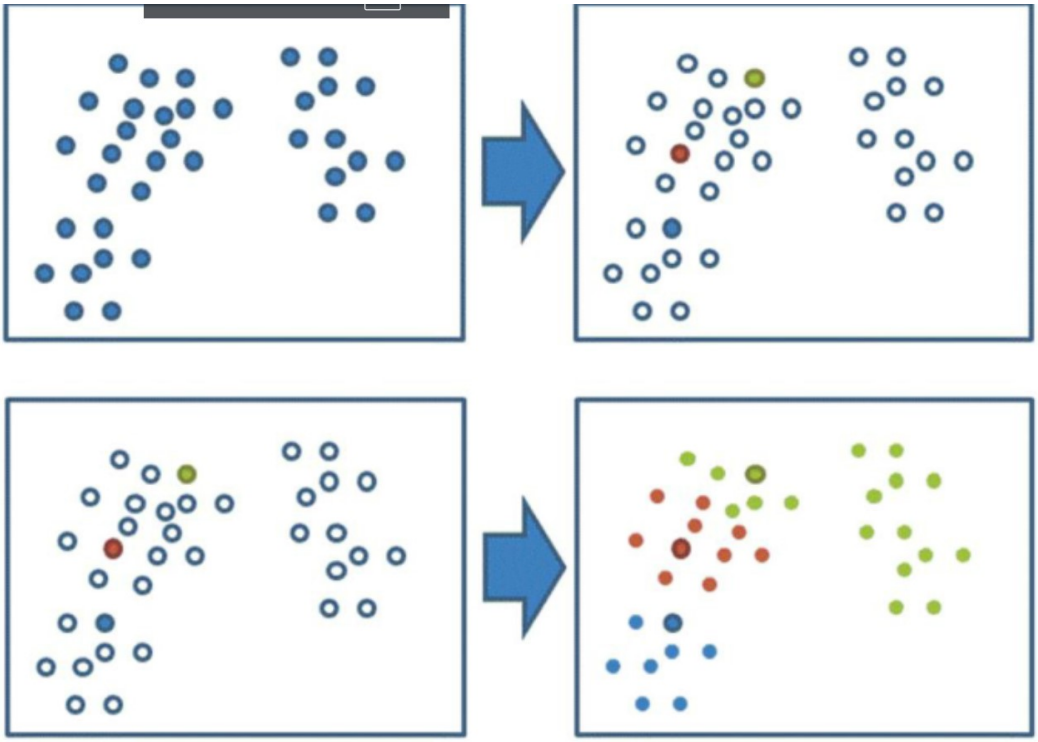
在如图的数据集中，我们需要把这些数据点分成两个部分，这就是我们所说的聚类任务。这也是一个无监督学习的任务，这需要知道数据的内容即可，不需要知道数据的标签。这里的数据比较分散，肉眼还比较好区分，但如果数据集十分密集的话，人类就无法靠自己的肉眼来对这些数据进行分别了。在日常的学习生活中，我们的数据是十分繁多的，数据点也是杂乱无章的，所以需要我们通过 K-Means 对数据进行聚类分析，进而分析我们的数据。



假设空间中有 N 个点，我们要把这 N 个点分成 K 类，且每个点都对应了一个 Z



这就是我们的运算的目标，使得每一个数据点和聚类中心的距离最小。算法的核心思想就是已知所有点的位置，但是不知道聚类中心的位置，通过计算两者之间的距离最小，经过一定数量的迭代次数，便可以找到那一个比较合适的聚类中心



上图便是一个 K-Means 算法的流程的简化示意图，通过不断的优化聚类中心，最终完成我们的聚类任务。

我们还采用了轮廓系数来描述模型的适合程度，轮廓系数接近于 1 说明效果最好，如果接近于 -1 则说明效果不好

$$sc_i = \frac{b_i - a_i}{\max(b_i, a_i)}$$

b_i为点i到其它簇的所有样本的距离最小值;
a_i为点i到本身簇的距离平均值。

2.3 K-Means 算法的主要步骤

1. 随机初始化质心

算法开始时，根据我们需要分成的数据的簇的数目 K，随机选择 K 个数据点作为初始质心，或者也可以采用其他方法来初始质心，进而加快算法的收敛

2.分配数据点到最近的质心

对于数据集中的所有数据点，计算他们与所有选择的 K 个质心的距离。将每一个数据点分配给离他们最近的质心，形成 K 个簇，通常使用欧氏距离来度量该距离的长短

3.更新质心位置

对于每一个簇，计算所有该簇内的数据点的均值，将该均值，也就是原簇内的数据点的中心位置，作为新的质心位置，将这 K 个簇的质心根据这一个操作进行更新

4.迭代

设置一个迭代次数，重复上述所说的第二步和第三步。直到质心的变化非常小即两次迭代之间的质心位置变化小于某个阈值、达到预设的迭代次数或者簇的分配不再发生变化

5.输出结果

输出最终的分配结果，每个数据点所属的簇，最终的簇的分配结果以及最终的质心位置，可以通过第三方库进行可视化操作

2.4 K-Means 算法的应用场景

1.一家广告平台需要根据用户相似的喜好和购买习惯将其分成不同的小组，以便为相关客户推荐广告

2.将多个语音片段中相似度符合设定阈值的归为同一个类别

3.一个数据科学团队需要降低一个大型数据集的维度的数量，以便简化建模和降低文件的大小

2.5 智慧金融的实现流程

1.读取数据集，准备开始

2.生成相关性热力图，查看相关性皮尔森系数，从而判断各个成分之间的影响关系的程度

3.由于数据集庞大可能会有数据损坏，所以要对文件中的缺失值和异常值进行检查

4.如果有错误，则通过代码对里面的数据进行修复

5.将数据集中的数据进行修复和修改后，将其转化为 DataFrame 或 NumPy 数组

6.进行 PCA 主成分分析，并获取累计方差图

7.通过累计方差图决定要保留的数据维度

8.对数据进行 PCA 主成分分析，并进行归一化处理

9.使用 K-Means 模型训练数据，循环 20 次，打印对应的轮廓系数，从而确定分成几组

10.根据上述得到的最优分组数，将该数值带入到程序中，获得这个时候的模型结果

11.对分组情况进行可视化处理，分析数据，指导银行的工作

3. K-Means 算法的实现

3.1 程序代码

```python
#导入必要的模块
import numpy as np#导入numpy库进行各种数学计算
import pandas as pd#导入pandas库进行各种数据分析
import matplotlib.pyplot as plt#导入matplotlib库进行数据的可视化操作
from sklearn.preprocessing import StandardScaler#导入StandardScaler库用于数据标准化，缩放至均值为0，方差为1
from sklearn.cluster import KMeans#导入Kmeans算法，用于将数据进行聚类
from sklearn.metrics import silhouette_score #导入silhouette_score进行轮廓系数的计算，从而评估聚类类数

#读取智慧金融的数据集
dataset = pd.read_csv("C:/Users/HP/Desktop/credit_card.csv")#利用pandas库读取csv文件

#查看数据集情况并进行一定的统计分析
dataset_description_df = dataset.describe()#假设dataset是一个DataFrame，describe用来生成描述性统计信息，包括均值、标准
#差、最小值、四分位数、中位数和最大值
print(dataset_description_df)#打印对应的信息

#查看相关性-热力图
import seaborn as sns#导入seaborn库，基于matplotlib的数据可视化库
numerical_dataset=dataset.drop(columns=['CUST_ID'])#由于ID无用，所以从dataset中删除了这一列
corrmat=numerical_dataset.corr()#计算所有数值型列的相关性矩阵
plt.figure(figsize=(15, 15))#设置图形的大小为15*15英寸
plt.xticks(rotation=90)#将x轴刻度标签旋转90度
plt.yticks(rotation=90)#将y轴刻度标签旋转90度
sns.heatmap(corrmat, vmax=1.0, square=True)#使用heatmap绘制相关性矩阵的热力图，vmax设置颜色条的最大值，True使每个相关系数
#占据一个正方形
plt.show()#显示图形

#查看相关性-皮尔森系数
corr_data = numerical_dataset.corr(method='pearson')#从DataFrame中计算相关系数，1为完全正相关，corr完成计算
print(corr_data)#打印皮尔森相关系数
```

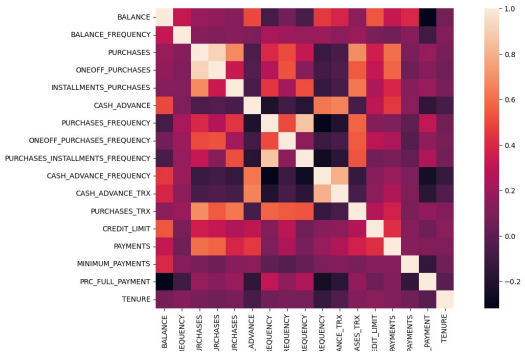
```python
#检查缺失数据
def get_missing_data_summary():  1 个用法
    dataset_na = dataset.isnull().sum()#计算数据集中每一列的缺失值数量
    dataset_na = dataset_na.drop(dataset_na[dataset_na == 0].index).sort_values(ascending=False)#移除没有缺失值的列
    #并按缺失值数量降序排序
    missing_data = pd.DataFrame({'Missing Count' :dataset_na})#将结果转化为一个DataFrame对象
    return missing_data#定义函数的返回值
missing_data = get_missing_data_summary()#获取数据的缺失数据
print(missing_data)#MINIMUM_PAYMENTS有313个缺失数据，CREDIT_LIMIT有1个缺失数据

#处理缺失数据
dataset=dataset.drop(dataset[dataset['CREDIT_LIMIT'].isnull()].index)#生成布尔序列，并获取缺失值的索引，丢弃缺失值并
#返回新的DataFrame对象
dataset=dataset.drop(dataset[dataset['MINIMUM_PAYMENTS'].isnull()].index)#重复上述操作，丢弃了另外一列的缺失值，并返回
#了新的对象，对dataset完成了所有的缺失值的处理

#移除无用的列
X=dataset.iloc[:,1:].values#使用iloc选择器从第二列开始选择所有，相当于移除第一列，并且用.values将DataFrame转化为Numpy数组

# 使用PCA降维
from sklearn.decomposition import PCA#导入PCA主成分分析的方法
pca = PCA(n_components = None)#创建新的PCA对象，并保留所有原始维度，计算每个主成分的方差比率
X_pca = pca.fit_transform(X)#拟合PCA模型，对numpy数组进行PCA变换，获得降维之后的数据X_pca
explained_variance_ratio = pca.explained_variance_ratio_#获取每一个主成分的方差比率
```

```python
#画出累计方差解释图，用于选择新生成的自变量的个数
plt.figure()#创建一个新的图形
plt.plot( *args: np.cumsum(pca.explained_variance_ratio_), c='orange')#绘制解释图，并将每个成分的方差比率求和展示
plt.xlabel('number of components')#设置x轴的标签为压缩后的数据维度
plt.ylabel('cumulative explained variance')#设置y轴的标签为累计方差
plt.show()#展示图形

# 选择新生成的自变量的个数
pca = PCA(n_components = 2)#指定保留两个主成分
X_pca = pca.fit_transform(X)#对数据X进行PCA主成分分析

# 特征缩放
sc_X = StandardScaler()#初始一个标准化器对象，使其均值为0方差为1
X_scaled = sc_X.fit_transform(X_pca)#对主成分分析之后的数据进行特征缩放

# 训练K-Means模型并打印轮廓系数，确定K-Means中K的个数
for i in range(2, 21):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', n_init=10, max_iter=300, random_state = 0)#初始化聚类对象
    #分组数等于i，算法将进行10次并选取最优结果，最大迭代次数为300
    kmeans.fit(X_scaled)#对标准化后的数据进行聚类分析
    y_kmeans = kmeans.predict(X_scaled)#预测数据点的聚类标签
    silhouette = silhouette_score(X_scaled, y_kmeans)#计算对应的轮廓系数
    print('当聚类个数是%d时，对应的轮廓系数是%.4f' %(i, silhouette))#打印对应的轮廓系数
```

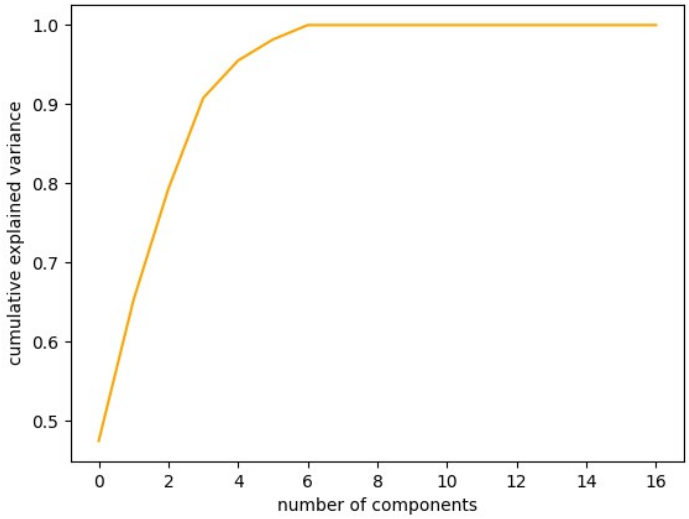
```python
#使用K=4建立K-Means模型
kmeans = KMeans(n_clusters = 4, init = 'k-means++', n_init=10, max_iter=300, random_state = 0)#初始化聚类对象
kmeans.fit(X_scaled)#对标准化后的数据进行聚类分析
y_kmeans = kmeans.predict(X_scaled)#获取预测点的标签

#可视化聚类效果
plt.figure()#创建新的图形
plt.scatter(X_scaled[y_kmeans == 0, 0], X_scaled[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 0')
#绘制散点图，绘制第1个聚类中两个主成分，以红色标记
plt.scatter(X_scaled[y_kmeans == 1, 0], X_scaled[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Cluster 1')
#绘制散点图，绘制第2个聚类中两个主成分，以蓝色标记
plt.scatter(X_scaled[y_kmeans == 2, 0], X_scaled[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Cluster 2')
#绘制散点图，绘制第3个聚类中两个主成分，以绿色标记
plt.scatter(X_scaled[y_kmeans == 3, 0], X_scaled[y_kmeans == 3, 1], s = 100, c = 'cyan', label = 'Cluster 3')
#绘制散点图，绘制第4个聚类中两个主成分，以蓝绿色标记
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 300, c = 'yellow', label = 'Centroi
#绘制每一个聚类的中心
plt.title('Clusters of customers')#设置图表题目
plt.xlabel('pca1')#设置x轴标签为pca1
plt.ylabel('pca2')#设置y轴标签为pca2
plt.legend()#显示图例
plt.show()#显示图表
```

3.2 实验结果

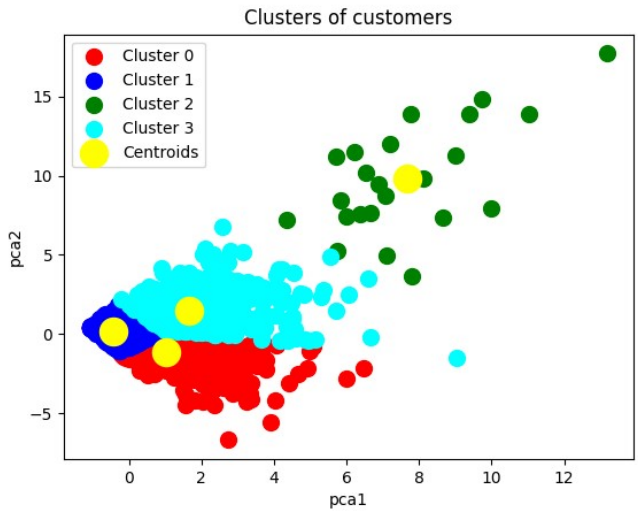
相关性热力图



累计方差图



聚类结果



原数据集的分析情况

```
D:\Conda\envs\Spider\python.exe C:\Users\HP\PyCharmMiscProject\.venv\1.py
            BALANCE  BALANCE_FREQUENCY  ...  PRC_FULL_PAYMENT       TENURE
count   8950.000000        8950.000000  ...       8950.000000  8950.000000
mean    1564.474828           0.877271  ...          0.153715    11.517318
std     2081.531879           0.236904  ...          0.292499     1.338331
min        0.000000           0.000000  ...          0.000000     6.000000
25%      128.281915           0.888889  ...          0.000000    12.000000
50%      873.385231           1.000000  ...          0.000000    12.000000
75%     2054.140036           1.000000  ...          0.142857    12.000000
max    19043.138560           1.000000  ...          1.000000    12.000000

[8 rows x 17 columns]
```

轮廓系数

```
当聚类个数是2时，对应的轮廓系数是0.5482
当聚类个数是3时，对应的轮廓系数是0.5592
当聚类个数是4时，对应的轮廓系数是0.5725
当聚类个数是5时，对应的轮廓系数是0.4795
当聚类个数是6时，对应的轮廓系数是0.4958
当聚类个数是7时，对应的轮廓系数是0.4263
当聚类个数是8时，对应的轮廓系数是0.4360
当聚类个数是9时，对应的轮廓系数是0.4307
当聚类个数是10时，对应的轮廓系数是0.4344
当聚类个数是11时，对应的轮廓系数是0.4094
当聚类个数是12时，对应的轮廓系数是0.4126
当聚类个数是13时，对应的轮廓系数是0.4107
当聚类个数是14时，对应的轮廓系数是0.4126
当聚类个数是15时，对应的轮廓系数是0.4048
当聚类个数是16时，对应的轮廓系数是0.4040
当聚类个数是17时，对应的轮廓系数是0.3972
当聚类个数是18时，对应的轮廓系数是0.3907
当聚类个数是19时，对应的轮廓系数是0.3898
当聚类个数是20时，对应的轮廓系数是0.3819
```

相关性皮尔森系数

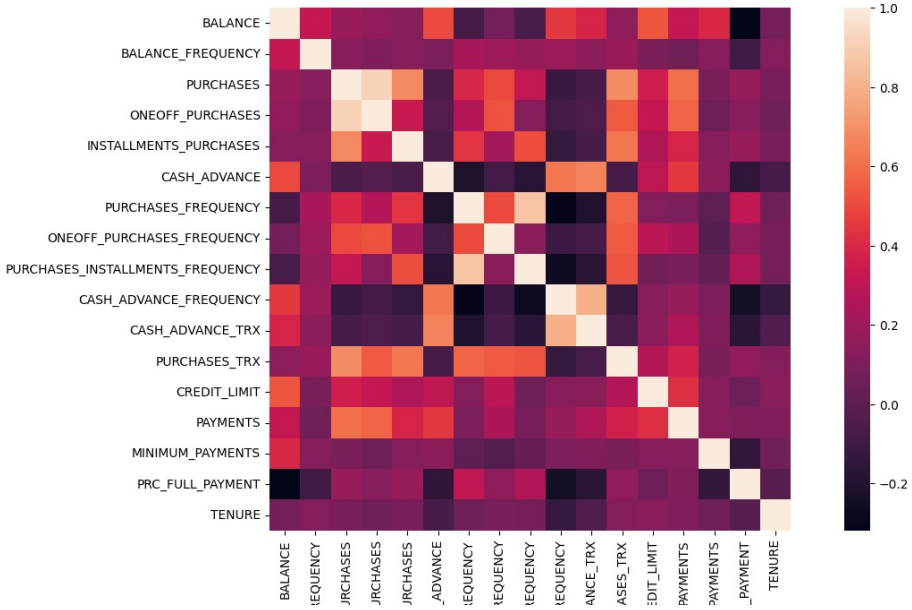
	BALANCE	...	TENURE
BALANCE	1.000000	...	0.072692
BALANCE_FREQUENCY	0.322412	...	0.119776
PURCHASES	0.181261	...	0.086288
ONEOFF_PURCHASES	0.164350	...	0.064150
INSTALLMENTS_PURCHASES	0.126469	...	0.086143
CASH_ADVANCE	0.496692	...	-0.068312
PURCHASES_FREQUENCY	-0.077944	...	0.061506
ONEOFF_PURCHASES_FREQUENCY	0.073166	...	0.082466
PURCHASES_INSTALLMENTS_FREQUENCY	-0.063186	...	0.073275
CASH_ADVANCE_FREQUENCY	0.449218	...	-0.133372
CASH_ADVANCE_TRX	0.385152	...	-0.043421
PURCHASES_TRX	0.154338	...	0.121874
CREDIT_LIMIT	0.531283	...	0.139167
PAYMENTS	0.322802	...	0.106136
MINIMUM_PAYMENTS	0.398684	...	0.059444
PRC_FULL_PAYMENT	-0.318959	...	-0.016486
TENURE	0.072692	...	1.000000

缺失数据

	Missing Count
MINIMUM_PAYMENTS	313
CREDIT_LIMIT	1

4. 总结分析

首先我们对相关性热力图中对我们展示的数据进行分析



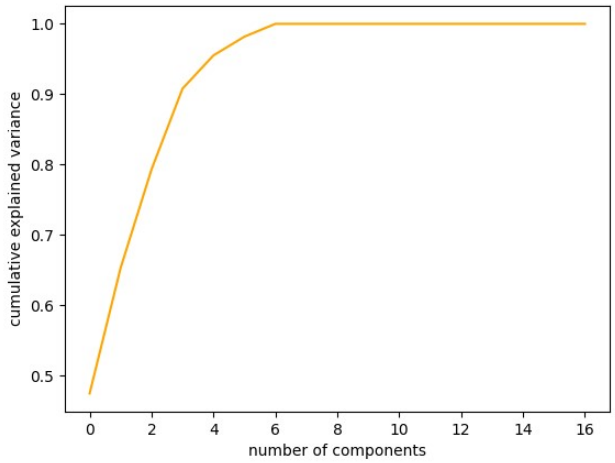
根据相关性皮尔森系数的定义，系数越接近于 1，则正相关性越强，越接近于-1，则负相关性就越强。所以根据图中的热力图，我们可以推断出相关性比较强的变量为单次最大消费金额和总消费额之间（颜色比较浅）等等的相关变量。对于颜色非常深的区块，我们可以粗略的估计为两者之间的关联性非常弱，所以在后续分析优质客户时需要优先考虑这些相关性皮尔森系数比较高的两组变量进行相互比较的时候，

在初次运行程序的时候报错，程序无法解析 csv 文件中的 ID 列，所以我增加了一行代码来解决这个问题：numerical_dataset=dataset.drop(columns=['CUST_ID'])

定义了新的 numerical dataset 移除了 ID 列

由于本人对相关性图的了解不深，这里这简单的对热力图进行了一个初步的解析

在接下来的任务当中，我们获取了累计方差图



但在这里我就产生了疑问，源代码中选择了保留两个主成分，但是通过这个图我们可以看出，两个主

成分只可以解释大概百分之七十的主成分，我们一般选择主成分的数量时，通常认为解释 90%左右的主成分是比较好的。所以带着这个疑问，我修改了源代码中的参数，分别设置了保留的主成分数目为 2，3，4，5，并输出了相应的轮廓系数进行了比较。因为我们知道，轮廓系数越接近于 1，则说明模型的分类情况更好。

保留 2 个主成分时

```
当聚类个数是2时，对应的轮廓系数是0.5482
当聚类个数是3时，对应的轮廓系数是0.5592
当聚类个数是4时，对应的轮廓系数是0.5725
当聚类个数是5时，对应的轮廓系数是0.4795
当聚类个数是6时，对应的轮廓系数是0.4958
当聚类个数是7时，对应的轮廓系数是0.4263
当聚类个数是8时，对应的轮廓系数是0.4360
当聚类个数是9时，对应的轮廓系数是0.4307
当聚类个数是10时，对应的轮廓系数是0.4344
当聚类个数是11时，对应的轮廓系数是0.4094
当聚类个数是12时，对应的轮廓系数是0.4126
当聚类个数是13时，对应的轮廓系数是0.4107
当聚类个数是14时，对应的轮廓系数是0.4126
当聚类个数是15时，对应的轮廓系数是0.4048
当聚类个数是16时，对应的轮廓系数是0.4040
当聚类个数是17时，对应的轮廓系数是0.3972
当聚类个数是18时，对应的轮廓系数是0.3907
当聚类个数是19时，对应的轮廓系数是0.3898
当聚类个数是20时，对应的轮廓系数是0.3819
```

保留 3 个主成分时

```
当聚类个数是2时，对应的轮廓系数是0.4371
当聚类个数是3时，对应的轮廓系数是0.4445
当聚类个数是4时，对应的轮廓系数是0.4493
当聚类个数是5时，对应的轮廓系数是0.4578
当聚类个数是6时，对应的轮廓系数是0.4670
当聚类个数是7时，对应的轮廓系数是0.3846
当聚类个数是8时，对应的轮廓系数是0.3952
当聚类个数是9时，对应的轮廓系数是0.3937
当聚类个数是10时，对应的轮廓系数是0.4038
当聚类个数是11时，对应的轮廓系数是0.4065
当聚类个数是12时，对应的轮廓系数是0.4068
当聚类个数是13时，对应的轮廓系数是0.3412
当聚类个数是14时，对应的轮廓系数是0.3149
当聚类个数是15时，对应的轮廓系数是0.3153
当聚类个数是16时，对应的轮廓系数是0.3011
当聚类个数是17时，对应的轮廓系数是0.3087
当聚类个数是18时，对应的轮廓系数是0.3170
当聚类个数是19时，对应的轮廓系数是0.3159
当聚类个数是20时，对应的轮廓系数是0.3232
```

保留 4 个主成分时

```
当聚类个数是2时，对应的轮廓系数是0.4056
当聚类个数是3时，对应的轮廓系数是0.4175
当聚类个数是4时，对应的轮廓系数是0.4154
当聚类个数是5时，对应的轮廓系数是0.4260
当聚类个数是6时，对应的轮廓系数是0.4274
当聚类个数是7时，对应的轮廓系数是0.4376
当聚类个数是8时，对应的轮廓系数是0.4429
当聚类个数是9时，对应的轮廓系数是0.3623
当聚类个数是10时，对应的轮廓系数是0.3624
当聚类个数是11时，对应的轮廓系数是0.3429
当聚类个数是12时，对应的轮廓系数是0.3451
当聚类个数是13时，对应的轮廓系数是0.3739
当聚类个数是14时，对应的轮廓系数是0.3777
当聚类个数是15时，对应的轮廓系数是0.2685
当聚类个数是16时，对应的轮廓系数是0.3020
当聚类个数是17时，对应的轮廓系数是0.2555
当聚类个数是18时，对应的轮廓系数是0.2708
当聚类个数是19时，对应的轮廓系数是0.2645
当聚类个数是20时，对应的轮廓系数是0.3000
```

保留 5 个主成分时

```
当聚类个数是2时，对应的轮廓系数是0.4470
当聚类个数是3时，对应的轮廓系数是0.3752
当聚类个数是4时，对应的轮廓系数是0.3883
当聚类个数是5时，对应的轮廓系数是0.3921
当聚类个数是6时，对应的轮廓系数是0.3992
当聚类个数是7时，对应的轮廓系数是0.4079
当聚类个数是8时，对应的轮廓系数是0.4132
当聚类个数是9时，对应的轮廓系数是0.4194
当聚类个数是10时，对应的轮廓系数是0.3508
当聚类个数是11时，对应的轮廓系数是0.3253
当聚类个数是12时，对应的轮廓系数是0.3264
当聚类个数是13时，对应的轮廓系数是0.3288
当聚类个数是14时，对应的轮廓系数是0.3292
当聚类个数是15时，对应的轮廓系数是0.2384
当聚类个数是16时，对应的轮廓系数是0.2566
当聚类个数是17时，对应的轮廓系数是0.2477
当聚类个数是18时，对应的轮廓系数是0.2301
当聚类个数是19时，对应的轮廓系数是0.2514
当聚类个数是20时，对应的轮廓系数是0.2352
```

通过打印以上数据我们可以归纳出：

保留 2 个主成分时，分成 4 组最好，对应轮廓系数为 0.5725

保留 3 个主成分时，分成 6 组最好，对应轮廓系数为 0.4670

保留 4 个主成分时，分成 8 组最好，对应轮廓系数为 0.4429

保留 5 个主成分时，分成 2 组最好，对应轮廓系数为 0.4470

我们选取轮廓系数最大的第一组，即保留两组主成分，而且保留两组主成分利于最后的可视化，因为如果保留的主成分超过 3，则可视化的维度超过了三维，我们无法进行感知。

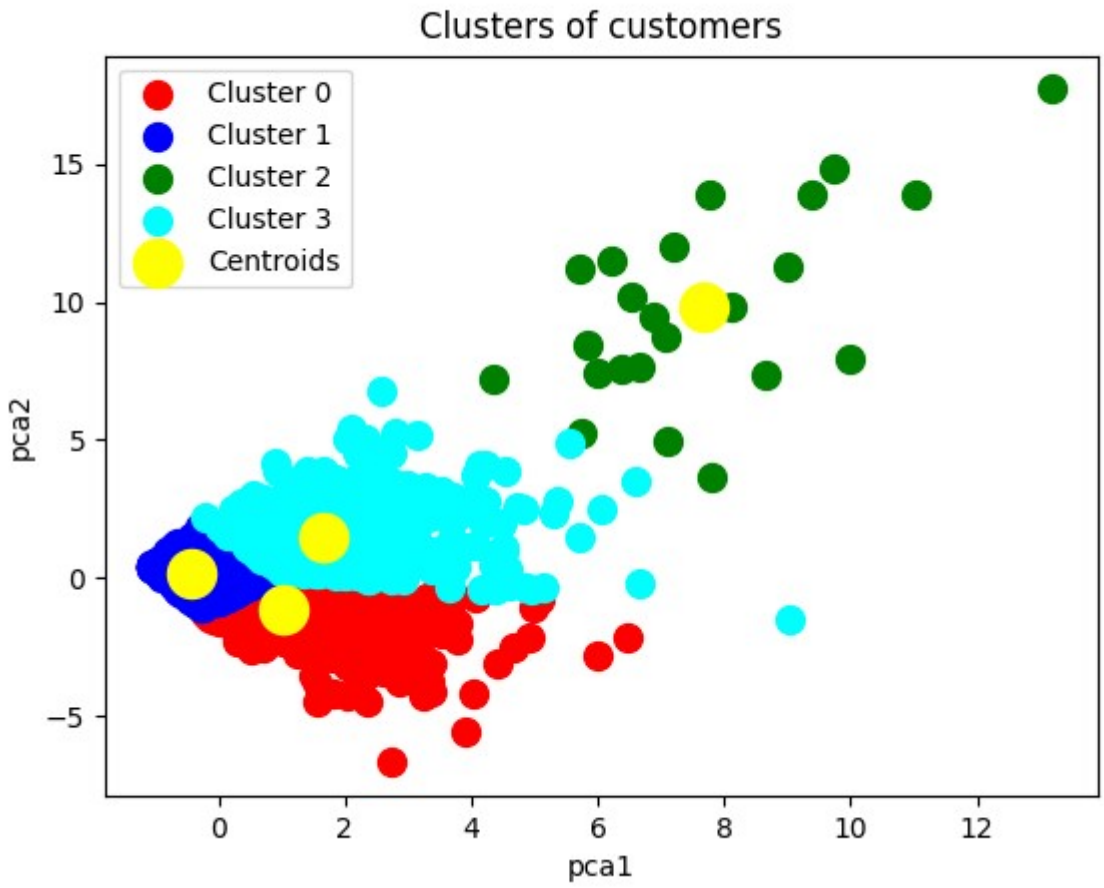
但是按照理论来讲，70%的主成分解释度并不是很好的解释了原数据，那么可能的原因可能存在以下几点：

1. 数据集在两个主成分上的分布可能更适合算法进行聚类。因为如果我们选取更多的主成分，我们无法保证我们学习到的特征点是数据集本身所带有的，所以额外的主成分可能包含的是噪音或者不重要的信息，才导致增加主成分数量后轮廓系数反而下降

2. 由于轮廓系数本身对簇内距离和簇间距离的平衡非常敏感，所以较少的主成分可能更好的分离了簇，从而提高了轮廓系数

3. 有时候，降维到两个主成分可能会意外的突出数据中的某些结构，这些结构在高维空间中可能不太明显，但在低维空间中却有助于聚类

所以，综上所述，虽然 3-5 个主成分更能比较好的显示数据集的原貌，但是出于轮廓系数等等的考虑，我们最终考虑设置降维时保留 2 个主成分，并且将数据集分成 4 个簇。



最后我们获得了这么一幅分类图

在源代码中，通过这个代码：

```
indices_of_0=[index for index, value in enumerate(y_kmeans) if value == 0]
print(indices_of_0)
indices_of_1=[index for index, value in enumerate(y_kmeans) if value == 1]
print(indices_of_1)
indices_of_2=[index for index, value in enumerate(y_kmeans) if value == 2]
print(indices_of_2)
indices_of_3=[index for index, value in enumerate(y_kmeans) if value == 3]
print(indices_of_3)
```

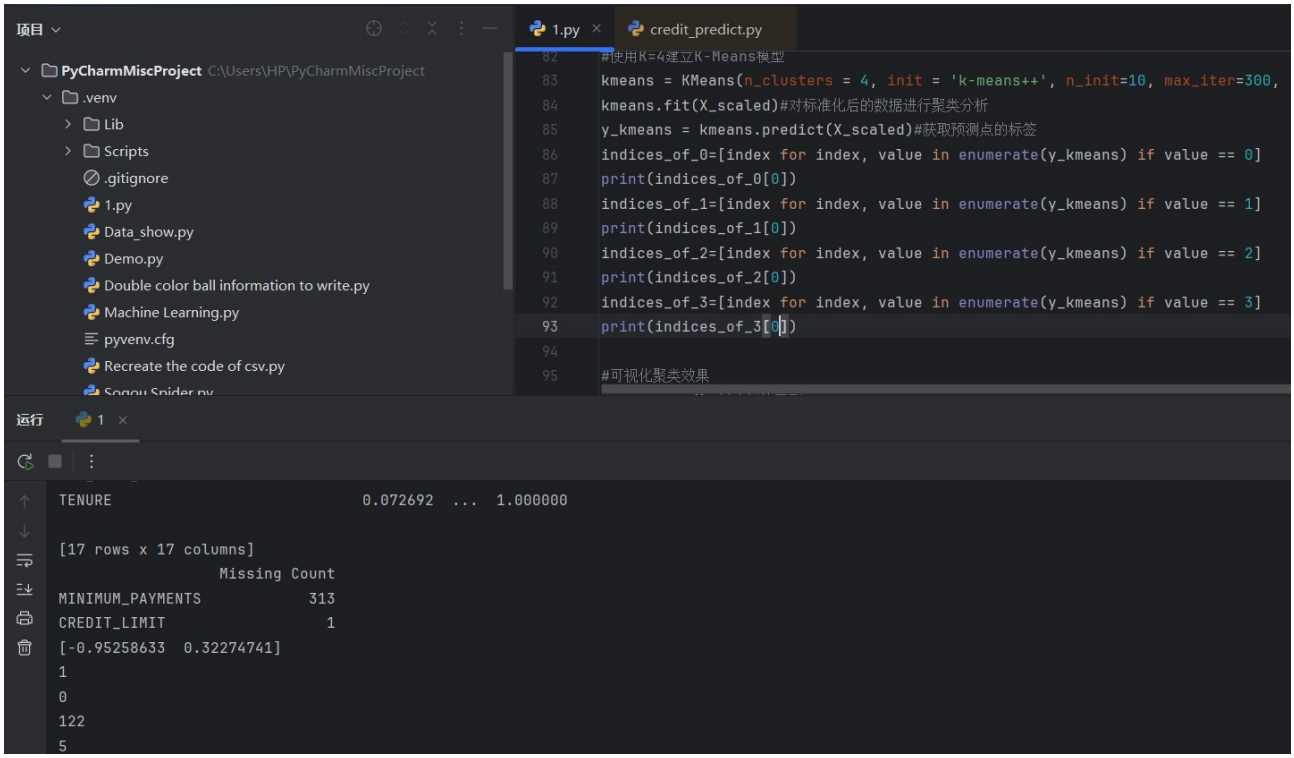
分别输出了四种分类的对应的索引，并通过内置函数 len()计算了每一类所包含的客户数量

红色 Cluster0 1726 人

蓝色 Cluster1 6367 人

绿色 Cluster2 23 人

青色 Cluster3 520 人



然后我们在每一个类别中都获取了一个用户的索引，单独拿出他们的信息，进行比较分析

索引分别是 1，0，122，5

所以对应 ID 分别是 2，1，123，6

1	C10001	40.90075	0.818182	95.4	0	95.4	0	0.166667	0	0.083333	0	0	2	1000	201.8021	139.5098	0	12
2	C10002	3202.467	0.909091	0	0	0	6442.945	0	0	0	0.25	4	0	7000	4103.033	1072.34	0.222222	12
3	C10006	1809.829	1	1333.28	0	1333.28	0	0.666667	0	0.583333	0	0	8	1800	1400.058	2407.246	0	12
4	C10123	54.93323	0.272727	495	0	495	0	0.166667	0	0.166667	0	0	2	7000	1969.846	147.05	1	12

通过对上述数据的分析，标签值为 3 的一类，也就是青色那一类的客户，他们流水很多，而且主要依赖分期还款，信用卡额度比较低，但是平常需要信用卡的时候比较多，他们需要提高自己的信用额度，同时经济状况处于中游水平，所以银行的目光可以着重放在这一些人身上，因为他们对于金钱的打理是很到

位的，很有可能是银行的潜在客户

标签值为 2 的一类，也就是绿色那一类的客户，他们的可用余额太少，虽然额度很高，但是他们使用信用卡的频率并不高，这一类客户信誉并不是特比好，不是特别靠谱，银行在推进业务中可以适当考虑抛弃这部分用户

标签值为 1 的一类，也就是蓝色的那一类，经济状况不太好，信用卡额度也不是特别的高，但是余额变动频率很高，平常用信用卡的几率还是很大的，可以适当的在他们身上下一些功夫

标签值为 0 的那一类，也就是红色的那一类，可用余额，余额变动频率，信用卡额度都很高，是实至名归的有钱人，且信誉良好，绝对是银行的大客户

综上所述，在上述的四分类中，我们推荐银行关注的客户（由高到低排序）：

红色>青色>绿色>蓝色

```
[1, 2, 8, 14, 17, 18, 23, 27, 29, 31, 33, 35, 37, 43, 47, 53, 63, 68, 69, 72, 74, 75, 78, 79, 80, 82, 83, 95, 99, 100, 107
```

这是节选的部分标签值为 0 的“最大优质客户的索引名单”，在实际对应中，他们的 ID 号是索引+1，银行可以根据这个名单去寻找优质客户

下面对实验误差进行分析：

1. 对缺失值和异常值的处理，通过程序我们可以发现，表格中还是存在缺失值的，对于采用众数，平均数等不同的方法填充表格后，得到的结果是不同的，所以对异常值的处理还是在很大的程度上影响了最后的实验误差
2. 再者就是运用 PCA 主成分分析的问题，PCA 或多或少会带来一定的数据损失，这个是无法避免的

改进的建议和一些其他的想法：

1. 可以采用更好的初始质点选取方法，因为该算法对此十分敏感，为了找到更好的质点中心，可以采用别的方法来初始化质心
2. 应该建立一个指标，在对数据进行降维和归一化之后，在特定的数值区间内，这个顾客就是什么样的类型客户，这样可以更直观的展示
3. 可以扩大数据的种类和数量，这样建立的模型解释度会更好